PROJECT HANDBOOK

Belin-Pok

Inventory, point of sale, online shop and owner reporting for a menswear retail & wholesale business.

Version: 24 October 2026

Covers: the API, the admin & inventory system, the web storefront and the phone app

Live at: belpok.xyz · admin.belpok.xyz · api.belpok.xyz

Contents

1 The system at a glance

What Belin-Pok does
The four parts
Who uses what
Principles behind the design

2 Running the shop — a guide for the owner and staff

Signing in
Staff roles
Products and stock
Prices: in-store, wholesale, pack, online
Selling at the till
Busy-hour sales
Voids, returns and corrections
Online orders
Customers and credit
Watching the shop from anywhere
Managing staff
Deleting test data
A daily routine

3 Technical reference

Architecture
Repository layout
The API
Data model
Sign-in and security
The web storefront
The admin system
The phone app
Configuration
API endpoint reference

4 Hosting and operations

Railway services
How a change goes live
Local development
Tests and checks
Common tasks
Backups and recovery

5 Status and next steps

Outstanding setup
Known limitations
Recommended next features

1 The system at a glance

Belin-Pok runs a menswear shop that sells caps, tees, jackets, sweatshirts, shorts, joggers, boxers and socks — over the counter to walk-in customers, in bulk to businesses buying wholesale, and online. One system keeps all three in step.

1.1 What Belin-Pok does

- **Stock:** every size and colour of every product is tracked separately. Deliveries are logged in seconds, and stock comes off automatically when anything sells, in store or online.
- **Point of sale:** a till for the counter, with retail, wholesale and factory-pack prices, split payments (cash, mobile money, card, on account), and shortcuts for busy moments.
- **Online shop:** a full website (and a phone app) where customers browse, order for pickup or delivery, and track orders. Online prices carry a configurable markup over in-store prices.
- **Wholesale accounts:** approved business customers can buy on credit up to a limit, with a complete ledger of charges and payments.
- **Owner oversight:** sales reports for any day, week or period; a receipt-by-receipt sales journal; and an activity log of everything staff did — built for an owner who isn't always in the shop.

1.2 The four parts

Part	Address	What it is
Admin & inventory system	admin.belpok.xyz	Where the shop is run: the till, stock, products, orders, customers, reports and staff. Works on a computer, tablet or phone.
Web storefront	belpok.xyz	The public online shop. Fast, searchable, and shareable — product links show the photo and price on WhatsApp and social media.
Phone app	Android / iOS (to be published)	The storefront as an installable app, with push notifications for order updates and back-in-stock alerts.
API	api.belpok.xyz	The engine behind all three: data, rules, prices, sign-in and permissions. Nothing reaches the database except through it.

1.3 Who uses what

Person	Uses	Typical tasks
Owner	Admin (everything)	Reports, journal, activity log, pricing, staff, approving wholesale accounts
Sales staff	Admin: till, orders, customers	Ringing up sales, packing online orders, taking payments on account

Inventory manager	Admin: products, stock, till, reports	Receiving deliveries, pricing and publishing products, stock counts
Retail customer	Website or phone app	Browsing, ordering, tracking orders, restock alerts
Wholesale customer	In person (till), with an account	Buying in bulk at wholesale/pack prices, on credit if approved

1.4 Principles behind the design

Nothing changes without a record

Stock never changes by editing a number: every change is a *movement* (received, sold, corrected, returned) with who made it and why, so the history always adds up to what's on the shelf. Customer credit works the same way. Mistakes are fixed with offsetting entries (a void, a correction), not by erasing history. Only an owner can delete outright, and even that leaves a note in the activity log.

The server decides prices and permissions

Screens show prices and hide buttons for convenience, but the API re-prices every sale and order itself and checks every request against the person's role. A tampered request can't sell below price, oversell stock, or do something a role doesn't allow.

Built for a real counter

Busy hours, staff without barcode scanners, phones and tablets at the counter, mobile money and buying on account — the till has shortcuts for all of them (quick log by type, one-amount baskets broken down later, split payments) without losing accuracy.

2 Running the shop

A practical guide to the admin system at **admin.belpok.xyz**, organised by task. What each person sees depends on their role; anything they can't do is simply not shown.

2.1 Signing in

- Sign in with an **email address**, a **phone number** (for staff invited by phone — type it any way, e.g. `024 123 4567` or `+233 24 123 4567`), or a **username** (the first owner account).
- Sessions stay signed in on that device and renew themselves; changing your password signs you out everywhere else.
- After 10 wrong passwords in 15 minutes, that account is paused for 15 minutes.
- **Forgot your password?** Staff with an email can request a reset link (valid for 1 hour). Anyone else asks an owner for a new sign-in link from *Staff*.

2.2 Staff roles

A person can hold several roles and gets everything each allows — for example, inventory manager *and* sales. Owner and Viewer stand alone.

Can…	Owner	Inventory manager	Sales & orders	Viewer
Use the till, void sales	✓	✓	✓	—
Receive stock, adjust stock	✓	✓	adjust only	—
Create & edit products, prices, photos, categories	✓	✓	—	—
Dashboard, sales reports, sales journal	✓	✓	—	—
Update online orders	✓	—	✓	—
Customers, wholesale approval, credit payments	✓	—	✓	—
View products, stock, orders, customers	✓	✓	✓	✓
Activity log, change history, staff, shop settings, deleting records	✓	—	—	—

2.3 Products and stock

Receiving a delivery

Receive stock (on Products, Inventory or a product's page) is the fast path. It stays open after each item so a whole delivery can be logged in one go.

- **New item:** take a photo, type a name and how many came in. It's saved as a **draft** priced at ₵0 — not on sale anywhere yet. Category, price, size and colour can be filled in now or later.
- **Already in the catalogue:** search for it, pick the size/colour (or add a new one), enter the quantity.

A product's life

Status	Meaning
Draft	Being set up. Not on the website, app or till. Fast-received items start here.
Live	Published: on the website, the app and the till. Publishing needs a price.
Archived	Discontinued: off the website and app. Leftover stock can still be sold at the till.

Sizes, photos and stock levels

- Each **size/colour** is its own SKU (generated automatically) with its own stock, prices and low-stock alert level (default 5).
- **Photos:** upload several at once; choose the cover photo shoppers see first. JPEG, PNG or WebP.
- **Inventory** shows stock for every SKU, what's running low (the reorder list), and the full movement history.
- **Adjust** records a restock, a count correction, a customer return, or a sale made outside the till — always with a reason.
- **Repack** records loose units organised into factory-style packs (a note; the count doesn't change).

2.4 Prices: in-store, wholesale, pack and online

Price	Where it's set	Who pays it
In-store	The product's price, or a size's own price	Walk-in customers at the till
Wholesale	Per size (blank = in-store price)	Wholesale customers at the till
Pack	Per size: a price for a whole factory pack, and pieces per pack	Wholesale customers buying sealed packs
Online	Automatic: in-store price + the shop's online markup (*Shop settings*), rounded up to a tidy amount — unless a size has its own fixed online price	Customers on the website and app

> **Set the online markup**
> The markup starts at **0%**, so online and in-store prices are the same until an owner sets it under **Shop settings**. The page shows a worked example table before saving. Changes reach checkout at once and the website's pages within a minute. Orders already placed keep their price.

Wholesale and pack prices are never shown to the public: the website and app only ever receive online prices.

2.5 Selling at the till

The **Point of sale** page has the product grid (search and category chips), the current sale, and today's receipts below.

1. Choose **Retail** or **Wholesale**. Wholesale asks for the customer first and unlocks wholesale and pack prices.
2. Tap products to add them: pick the size (sold-out sizes are struck through), the price tier, and the quantity. The till won't add more than is in stock.
3. Press **Charge**. Enter how the customer paid — split across several methods if needed (e.g. part cash, part mobile money). **On account** appears only for approved wholesale customers and only up to their available credit.
4. **Complete sale.** The API re-prices every line, checks stock and takes it off, and records any credit charge — all at once, or not at all.

2.6 Busy-hour sales

Two shortcuts keep the queue moving when finding exact products would slow things down. Both are tidied up later from **To sort out** under Today's sales; until then their stock hasn't come off, and the dashboard shows a reminder.

Quick log by type

Record a line by type of item ("Caps"), price and quantity. Later, **Identify** attaches the exact product (stock comes off then), or **Split** it if it was really several different products.

One amount

Take the money for a whole mixed basket as a single amount, with an optional note ("2 tees, a cap and socks"). Later, **Break down** lists what was in it — exact products, or just types — and only saves once the parts add up to exactly what was taken.

2.7 Voids, returns and corrections

Situation	What to do
A sale rung up wrongly	**Void** it (Today's sales or the journal) with a reason. Stock goes back and any credit charge is reversed; the sale stays in the journal marked voided, with who, when and why.
A customer returns goods	**Adjust stock** → **Customer return** on that size.
Stock count doesn't match	**Adjust stock** → **Correction** with the difference and a reason.
A customer's balance is wrong	On their page: **Correction** (up or down, with a reason).
Test data, or something that should never have existed	Owner only: **Delete** (see 2.12).

2.8 Online orders

- Orders from the website and app land as **Pending**. No payment is taken online: the shop calls the customer to confirm and arrange payment, pickup or delivery.
- Move each order along — **Paid** → **Packed** → **Shipped** → **Delivered** — with one click, adding a note (e.g. a mobile money reference) if useful. Every change is recorded in the order's timeline and the activity log.
- **Cancelling** puts the order's items back in stock, and so does **refunding** an order that hadn't been shipped yet. A refund after shipping leaves stock alone — record a customer return when the goods actually come back. Reopening a cancelled order takes the stock again, if it's still there.
- Each line shows the online price charged and what it would have cost in store; the order shows the markup it earned.
- A warning panel lists orders needing more stock than the system shows — usually a counter sale that wasn't rung up. Check the shelf before packing.
- Customers using the phone app get a notification when their order moves on (once push notifications are configured — see 5.1).

2.9 Customers and credit

- Customers are created when someone signs up on the website or app, when a guest places an order, or by staff (**Add customer**) — typically wholesale businesses.
- A wholesale account must be **approved** before it can buy on credit. Set its **credit limit** (0 = cash only).
- Each wholesale customer has a **credit account**: owes / limit / available, a usage bar (red past 85%), and every charge, payment and correction with who recorded it.
- **Record payment** when they pay something off; the dashboard lists who owes the most and for how long (red past 30 days).

2.10 Watching the shop from anywhere

Page	Use it for
Dashboard	The last 7, 30 or 90 days at a glance: revenue, items sold, credit owed, low stock, best sellers, latest stock activity, and a reminder of sales still to sort out.
Sales reports	Today's end-of-day figures, or any period (presets or custom dates) by day, week or month. Where the money came from (cash — what should be in the drawer — mobile money, card, on account, online), voids, credit given and collected, and breakdowns by product, staff member and category. Printable and exportable to CSV.
Sales journal	Every sale receipt by receipt: when, who rang it up, the customer, each line, how it was paid, and void details. Switch between in-store and online. Filter by period, staff, payment method or status; export every line to CSV.
Activity log owner	Everything staff did, in one timeline: sales, voids (with reasons), stock changes, product and price edits, order updates, payments on account, staff changes, breakdowns and deletions. Filter by period, person or kind.

Days run midnight to midnight in Ghana time (Africa/Accra). In-store sales count unless voided; online orders count unless cancelled or refunded.

2.11 Managing staff owner

1. **Staff → Invite staff.** Enter their name, an email and/or a phone number, and tick their roles.
2. Choose how to send the invite: **Email** (sent automatically when email is set up, otherwise your email app opens), **WhatsApp**, **SMS**, your phone's **Share** menu, or **Copy**. The message is written for you, including the link and what to sign in with.
3. They open the link (valid once, for 7 days), choose a password, and they're in. Someone invited by phone only signs in with that number.

- **Change roles** with the pencil next to someone's roles. **Switch someone off** with the Active toggle: they can't sign in, but their history stays. Owners can't change their own roles or switch themselves off.
- **Sign-in link** issues a fresh link — for a forgotten password or an expired invite — with the same sending options.

2.12 Deleting test data owner

For records that should never have existed — a sale rung up while trying the till, a test order — an owner can delete outright. A reason is required; the activity log keeps a red **Deleted** entry saying what it was, who deleted it and why.

Record	Where	What happens
Till sale	Sales journal (open the receipt) or Today's sales	Gone from the journal and reports; stock back; credit charge off the customer's balance.
Online order	The order's page	Gone from orders and reports; its items back in stock.
Customer	The customer's page	Only once they have no orders, sales or credit history. Addresses and website login go too.

Stock entry	Inventory → Movement history	Only entries made by hand; their effect on stock is undone.
Product	The product's page	Including its stock history — but not while any sale or order includes it.

Deletion is refused, with an explanation, where undoing isn't possible: credit already paid off, received stock since sold, or a product still in a sale.

2.13 A daily routine

Opening & during the day

- Check the dashboard: low stock, orders needing attention.
- Log deliveries with Receive stock as they arrive.
- Move online orders along as they're paid and packed.
- Use quick log / one amount when busy.

Closing

- Sort out today's quick-logged and one-amount sales.
- Sales reports → Today: count the drawer against *Cash*.
- Review voids and their reasons.
- Owner: skim the activity log.

3 Technical reference

How the system is built, for whoever maintains it. The code is commented throughout; this chapter is the map.

3.1 Architecture

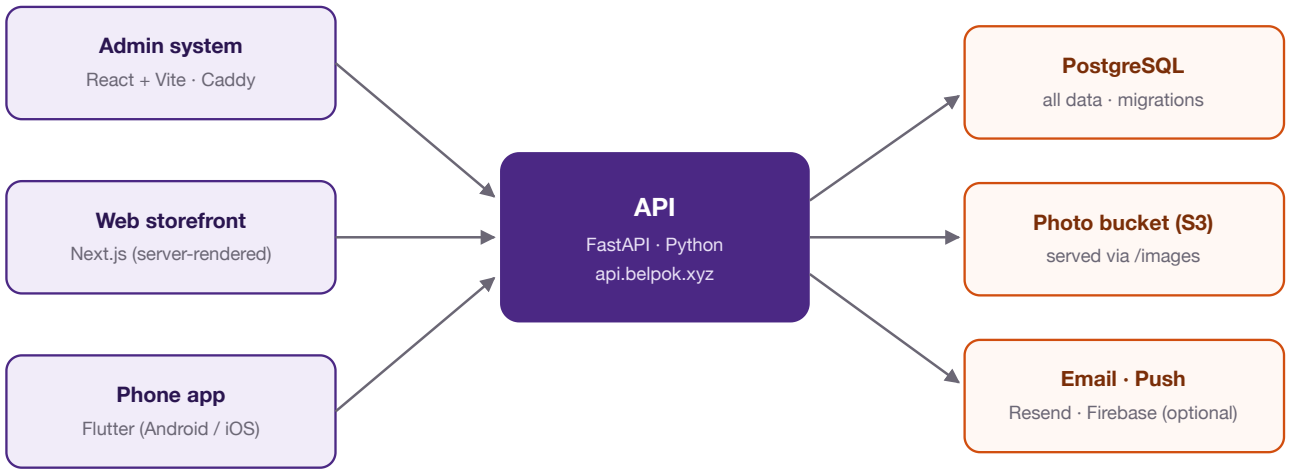


Figure 1 — Every app talks to the API over HTTPS; the API owns the data and the rules.

3.2 Repository layout

```
services/api/            FastAPI backend
  app/                   application code (routers/, models.py, schemas.py, ...)
  migrations/            SQL migrations, applied in filename order
  tests/                 pytest suite (138 tests)
apps/admin-web/          Admin & inventory system (React + Vite + Tailwind)
apps/web/                Web storefront (Next.js + Tailwind)
apps/storefront/         Phone app (Flutter)
packages/belpok_core/    Shared Dart package for the phone app
deploy/Caddyfile         Serves the built admin app
.github/workflows/       CI: lint, type-check and tests on pull requests
```

3.3 The API

Python 3 with **FastAPI**, **SQLAlchemy 2** (async, asyncpg driver) and **Pydantic**. Key files in `services/api/app/`:

File	Responsibility
`main.py`	Creates the app, CORS, registers every router.
`models.py` / `schemas.py`	Database tables (ORM) / request and response shapes.

`deps.py`	Who is calling: resolves the token to a staff member or customer; `require_role` guards.
`security.py`, `routers/auth.py`	Passwords, tokens, sessions, invite/reset links, sign-in throttling.
`pricing.py`	In-store vs online prices; what the public catalogue may see.
`erase.py`	Owner deletions: undoing stock and credit, recording the deletion.
`shop_time.py`	Shop-timezone day boundaries for reports.
`phone.py`	Normalising phone numbers for staff sign-in.
`storage.py`, `email.py`, `push.py`	Photo bucket, transactional email (Resend), push notifications (Firebase).
`migrate.py`, `cli.py`	Migration runner; command-line tools (creating the first owner).

Interactive API documentation is generated automatically at `/docs` on any running API.

3.4 Data model

24 tables in PostgreSQL, created by 34 SQL migrations. The main groups:

Area	Tables	Notes
People & sign-in	`users`, `staff`, `refresh_tokens`, `one_time_tokens`, `failed_logins`	A user is the login; staff and customers point to it.
Catalogue	`categories`, `products`, `product_variants`, `product_images`, `suppliers`	Variants (size/colour) hold stock and prices.
Stock	`stock_movements`	Append-only ledger; a trigger keeps `stock_quantity` in step and refuses negative stock.
Till	`pos_sales`, `pos_sale_items`, `pos_sale_payments`	Lines snapshot the price charged.
Online	`orders`, `order_items`, `order_status_history`	Lines record online and in-store price.
Customers & credit	`customers`, `addresses`, `customer_credit_ledger`, `customer_device_tokens`, `stock_notification_requests`	Ledger trigger keeps the balance and enforces the credit limit.
Oversight & settings	`audit_log`, `shop_settings`	Before/after of admin changes; online pricing.

Database triggers are the last line of defence: even if API code had a bug, stock can't go negative and a credit sale can't exceed a customer's limit.

3.5 Sign-in and security

- Passwords are hashed with **argon2**. Access tokens last 15 minutes; refresh tokens last 30 days, are single-use and rotate on every refresh — reuse of an old one signs that person out everywhere.
- Invite links last 7 days, reset links 1 hour; both work once. Sign-in pauses after 10 failures in 15 minutes.
- Every staff request checks the person is still active, so switching someone off takes effect immediately.
- The public catalogue endpoints return only live products and online prices — never wholesale, pack or in-store prices.
- Secrets live only in Railway's service variables, never in the repository.

3.6 The web storefront (`apps/web`)

Next.js 16 with React 19 and Tailwind CSS 4, running as a Node server. Catalogue pages (home, shop, categories, products) are rendered on the server and cached for a minute, so they're fast, indexed by search engines, and show rich previews when shared. Cart, checkout and accounts run in the browser; the cart is kept on the shopper's device.

Pages: home, shop (search, filters, sorting — all in the URL), category, product, cart, checkout (guest or signed in; pickup or delivery), order confirmation, account (orders and addresses), sign-in, register, password reset, wholesale, sitemap and robots.

3.7 The admin system (`apps/admin-web`)

React 19 + Vite, React Router, TanStack Query (data caching and refresh), Recharts, Tailwind CSS 4. Built to static files served by Caddy. Pages live in `src/pages/`, shared components in `src/components/`, and the API client, session and helpers in `src/lib/`. Data refreshes when the window regains focus, so a tablet left at the counter catches up with sales made elsewhere.

3.8 The phone app (`apps/storefront`)

Flutter, with shared models, API client and session in `packages/belpok_core`. Browsing, cart, checkout and order history against the same API, plus push notifications. Android and iOS projects are configured (app ID `com.belpok.storefront`); publishing needs your Play Console and Apple Developer accounts — see the README's release section.

3.9 Configuration

API (Railway service variables)

Variable	Purpose
`ENVIRONMENT`	`production` on Railway; `local` for development.
`DATABASE_URL`	Postgres, with the `postgresql+asyncpg://` scheme.
`JWT_SECRET`	Signs access tokens. Long and random; changing it signs everyone out.
`S3_ENDPOINT_URL`, `S3_BUCKET`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `S3_REGION`	The product photo bucket.
`CORS_ORIGINS`	Every address the web apps are served from.
`ADMIN_APP_URL`, `STOREFRONT_URL`	Where invite and password-reset links point.
`SHOP_TIMEZONE`	Report day boundaries (default `Africa/Accra`).
`RESEND_API_KEY`, `EMAIL_FROM`	Optional: sending invite and reset emails.
`FIREBASE_SERVICE_ACCOUNT_JSON`	Optional: push notifications to the phone app.
`ACCESS_TOKEN_MINUTES`, `REFRESH_TOKEN_DAYS`	Optional session lengths (defaults 15 and 30).

Web apps (build-time — changing one needs a redeploy)

Variable	Purpose
`NEXT_PUBLIC_API_BASE_URL`	Storefront: the API address.
`NEXT_PUBLIC_SITE_URL`	Storefront: its own address, for links and the sitemap.
`NEXT_PUBLIC_CONTACT_PHONE`, `…_WHATSAPP`, `…_EMAIL`, `NEXT_PUBLIC_SHOP_ADDRESS`	Storefront: optional contact details for the footer and wholesale page; unset ones are left out.
`VITE_API_BASE_URL`, `VITE_STOREFRONT_URL`	Admin: the API, and where "View on the shop" links go.

3.10 API endpoint reference

77 endpoints. "Staff" means any signed-in staff member; role limits follow section 2.2. Full request/response details are at `/docs`.

Sign-in		
POST	`/auth/login · /auth/refresh · /auth/logout`	Start, renew, end a session
POST	`/auth/signup`	Customer creates an account
POST	`/auth/forgot-password · /auth/set-password`	Reset link; set a password from a link
POST	`/auth/change-password`	Change own password (signs out other devices)
Public catalogue (no sign-in; online prices only)		
GET	`/public/categories · /public/products`	Categories; products with search, filters, sorting
GET	`/public/products/{id} · /public/products/by-slug/{slug}`	One product
GET	`/images/{path}`	Product photos
Products, stock and categories		
GET	`/products · /products/{id}`	Catalogue (all statuses)
POST	`/products · /products/receive`	Create a product; receive stock (fast path)
PATCH	`/products/{id}`	Edit, publish, archive
DELETE	`/products/{id}`	Delete (owner may include stock history)
GET / POST	`/products/{id}/variants · /products/{id}/images`	Sizes/colours; photos
PATCH	`/variants/{id}`	A size's prices, alert level, visibility
GET	`/variants · /variants/low-stock`	Inventory list; reorder list
POST	`/variants/{id}/notify-when-in-stock`	Customer restock alert
PATCH / DELETE	`/images/{id}`	Make cover photo; delete photo
POST	`/uploads/product-photo`	Upload a photo to the bucket
GET / POST	`/stock-movements`	Movement history; record a movement
DELETE	`/stock-movements/{id}`	Owner: delete a hand-made entry
POST	`/inventory/rebalance`	Record repacking into packs
GET / POST / PATCH / DELETE	`/categories · /categories/{id}`	Manage categories
Till		
GET / POST	`/pos-sales`	Journal (filters, `needs_details`); ring up a sale
POST	`/pos-sales/{id}/void`	Void with a reason

POST	/pos-sales/{id}/items/{item}/identify · …/break-down	Name a quick-logged line; split a line into parts
DELETE	/pos-sales/{id}	Owner: delete a sale
Online orders		
POST	/orders/checkout	Place an order (guest or signed in)
GET	/orders · /orders/{id} · /orders/me · /orders/needs-attention	Staff list; one order; a customer's own; stock warnings
POST	/orders/{id}/status	Move an order along
DELETE	/orders/{id}	Owner: delete an order
Customers and credit		
GET/POST	/customers · /customers/{id}	List, add, view
PATCH/DELETE	/customers/{id}	Edit; owner: delete
POST	/customers/{id}/status	Approve or reject
GET/POST	/customers/{id}/credit-ledger · credit-payments · credit-adjustments	Credit history; payments; corrections
GET/POST	/customers/me · /customers/me/addresses · /customers/me/device-tokens	A customer's own profile, addresses, devices
Reports, staff and settings		
GET	/reports/dashboard · /reports/daily-sales · /reports/sales	Dashboard; daily chart; full sales report
GET	/reports/activity · /audit-log	Owner: activity log; change history
GET/PATCH	/staff · /staff/me · /staff/{id}	Staff list; own record; roles and active
POST	/staff/invite · /staff/{id}/invite-link	Invite (email and/or phone); new sign-in link
GET/PATCH	/settings	Online pricing (change: owner)
GET	/health	Health check for Railway

4 Hosting and operations

4.1 Railway services

Everything runs in one Railway project, `talented-adaptation`, deployed from the `main` branch of the GitHub repository.

Service	Address	Built from
`belin-pok`	api.belpok.xyz	`services/api` (Nixpacks; `railway.json` sets start command, health check and migrations)
`web`	belpok.xyz, www.belpok.xyz	`apps/web/Dockerfile`
`admin-web`	admin.belpok.xyz	`apps/admin-web/Dockerfile` (repo root as context, for `deploy/Caddyfile`)
`Postgres`	private network only	Railway database
`product-photos`	private	Railway bucket, served through the API

The web services also answer on `web-production-a8a25.up.railway.app` and `admin-web-production-e077.up.railway.app`. belpok.xyz's DNS is managed in Cloudflare, which also proxies the traffic; the apps themselves are hosted on Railway. Invite and reset links point at admin.belpok.xyz and belpok.xyz.

4.2 How a change goes live

1. Commit and push to `main`.
2. Railway rebuilds only the services whose files changed (watch paths): API, storefront and admin are independent.
3. For the API, the pre-deploy step runs `python -m app.migrate` first. A failed migration stops the deploy, and the previous version keeps running.
4. The new version takes over once its health check passes (`/health` for the API).

Check progress with `railway deployment list --service <name>` or in the Railway dashboard. To roll back, redeploy an earlier deployment from the dashboard — but note a migration that already ran is not undone by a rollback.

4.3 Local development

```
# Database and API
createdb belpok
cd services/api && python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt && cp .env.example .env    # set DATABASE_URL, JWT_SECRET
python -m app.migrate
python -m app.cli create-owner --username admin --password admin
uvicorn app.main:app --reload                       # http://localhost:8000/docs

# Admin                                             # http://localhost:5173
cd apps/admin-web && npm install && VITE_API_BASE_URL=http://localhost:8000 npm run dev

# Storefront                                        # http://localhost:3000
cd apps/web && npm install && NEXT_PUBLIC_API_BASE_URL=http://localhost:8000 npm run dev
```

Without bucket settings, photos are kept in memory locally (lost on restart). Without email settings, invite links are logged and still shown in the admin.

4.4 Tests and checks

```
cd services/api
ruff check .                    # lint
python -m pytest -q             # 138 tests; needs DATABASE_URL to a migrated database
cd apps/admin-web && npm run build      # type-check + build
cd apps/web && npm run typecheck
```

GitHub Actions runs the API's lint and tests (against a fresh Postgres) and the web type-checks on pull requests. CI never deploys; Railway is the only deploy path.

4.5 Common tasks

Task	How
Create the first owner on a fresh database	Inside the API container (the database is private): `railway ssh --service belin-pok -- sh -c 'export "$(tr "\0" "\n" < /proc/1/environ | grep "^LD_LIBRARY_PATH=")" && cd /app && /opt/venv/bin/python -m app.cli create-owner --username admin --password admin'`
A staff member is locked out	Admin → Staff → **Sign-in link**, send it to them.
Change the schema	Add a new file `services/api/migrations/YYYYMMDDHHMMSS_what.sql`. Never edit one that has already run in production.
Change a web app setting	Edit the service's variable in Railway, then redeploy (they're compiled in).
Add a domain	`railway domain <domain> --service <service> --port 8080` prints the DNS records to add in Cloudflare. Then add it to `CORS_ORIGINS` on `belin-pok` (and `ADMIN_APP_URL` / `STOREFRONT_URL`, where invite and reset links point, if it replaces one).
Back up the database	`scripts/backup-db.sh` — see 4.6.

4.6 Backups and recovery

All business data lives in the Railway Postgres service; product photos are in the bucket.

> **Railway backups need a Pro workspace**
>
> Railway's scheduled database backups (and its one-click restore) are only available on its Pro plan, which this workspace isn't on — so there are no backups on Railway yet. Upgrading, then turning on daily, weekly and monthly backups for the `postgres-volume`, is the most reliable protection.

Your own backups, in the meantime

```
scripts/backup-db.sh        # saves ~/belpok-backups/belpok-YYYYMMDD-HHMM.sql.gz
```

Run it from the repository on a computer signed in to Railway — weekly, and before any risky change. It dumps the whole database inside Railway, downloads it, and refuses to keep a dump that stopped part-way. The files hold customer details, so keep them private; copy them somewhere off the computer (an encrypted drive or private cloud folder) too.

To restore a backup into a database (a fresh local one to inspect, or a new Railway database):

```
createdb belpok_restore
gunzip -c ~/belpok-backups/belpok-YYYYMMDD-HHMM.sql.gz | psql -d belpok_restore
```

The first backups were taken (and one test-restored) on 23–24 October 2026. Product photos aren't in it; the bucket is separate.

5 Status and next steps

Where things stand on 24 October 2026.

5.1 Outstanding setup

Step	Why it matters
Add two DNS records for www.belpok.xyz in Cloudflare, set up the same way as the existing `belpok.xyz` and `admin` records: `CNAME www → afz6kirc.up.railway.app` `TXT _railway-verify.www → railway-verify=acff4956ad71ee5f91d5846aa7ec294c9e2ff49f291689823f69215b4a0d1dc3`	The domain is attached to the storefront on Railway, but has no DNS yet, so typing "www.belpok.xyz" currently fails. belpok.xyz itself works.
Database backups: upgrade the Railway workspace to Pro, then turn on daily/weekly/monthly backups for the Postgres volume	None exist on Railway yet — its scheduled backups need a Pro workspace. Until then, run `scripts/backup-db.sh` regularly (see 4.6).
Set the online markup (Admin → Shop settings)	It's 0% until set, so online prices equal in-store prices.
Email sending: a Resend account with belpok.xyz verified; set `RESEND_API_KEY`	Not configured. Invites can still go by WhatsApp/SMS, but customers can't reset forgotten passwords by email until this is done.
Push notifications: a Firebase project; set `FIREBASE_SERVICE_ACCOUNT_JSON`	Not configured. Needed for order updates and restock alerts in the phone app.
Shop contact details on the `web` service	Phone, WhatsApp, email and address appear in the footer and wholesale page only once set.
Publish the phone app	Needs your Play Console / Apple Developer accounts and signing keys.
Delete the old Supabase project	Nothing uses it any more.

5.2 Known limitations

- **No online payment.** Orders are confirmed and paid by arrangement with the customer.
- A one-amount basket counts as **1 item** in "items sold" until it's broken down.
- Issuing a new sign-in link doesn't cancel older unused links; each still works once until it expires (or until any of them is used).
- Wholesale customers buy in person; there's no self-serve wholesale ordering online.
- Cost prices aren't recorded, so the system reports revenue, not profit.

5.3 Recommended next features

In order of value for an owner monitoring the shop from a distance:

#	Feature	Benefit
1	Cost prices and profit	Record what stock cost when received; profit per product, day and staff member (owner-only).
2	End-of-day cash-up	Staff count the drawer at close; the system records any shortfall and who closed.
3	Discounts with reasons	Haggling recorded properly — who gave how much and why — instead of showing as mismatches.
4	Receipts	Printable receipts for a thermal printer, plus WhatsApp/SMS copies.
5	Returns linked to the sale	Refunds tied to the original receipt, closing a common way money goes missing.
6	Barcode labels and scanning	Faster till, fewer quick-logged lines.
7	Suppliers and reordering	Who you buy from, what you paid, reorder lists from low-stock levels.
8	Online payment	Mobile money/card at checkout (e.g. Paystack).

> **Tax**
>
> If the business is VAT-registered, check with an accountant what Ghana Revenue Authority E-VAT invoicing requires before building receipts and invoices.